

E-SCOUT: Efficient-Spatial Clustering-based Outlier Detection through Telemetry

Eduardo Ortega¹, Jonti Talukdar², Woohyun Paik, Fei Su³, Rita Chattopadhyay⁴, Krishnendu Chakrabarty¹

¹Arizona State University, ASU Center for Semiconductor Microelectronics

²Duke University, Department of Electrical and Computer Engineering

³Intel Corporation, Ann Arbor MI

⁴Intel Corporation, Chandler AZ

Abstract—Silicon lifecycle management (SLM) is needed to ensure silicon-product reliability and quality. Prior methods utilize off-chip solutions to identify malware, diagnose bugs, and characterize silicon health metrics. These methods do not explore hardware/software codesign for SLM. In this work, we present a new method called Efficient-Spatial Clustering-based Outlier detection through Telemetry (E-SCOUT) to monitor a chip's status through performance counters/sensors. E-SCOUT includes a compute- and memory-efficient unsupervised 32-bit floating point outlier detection mechanism implemented on-chip (E-SCOUT edge). In addition, it enhances on-chip outlier detection through unsupervised feature ranking based on the telemetry feature information entropy. We also provide microarchitectural recommendations to enable a hardware/software co-design of E-SCOUT edge. The proposed solution includes an end-to-end outlier-informed diagnosis model with real telemetry data (E-SCOUT cloud). All telemetry data is collected through the model-specific register space using open-source Linux tools and Intel's performance counter monitor. We capture the chip telemetry signatures of the PAMPAR benchmark suite in the presence of outlier events such as security attacks (e.g., Rowhammer and SPECTRE) and voltage droops. E-SCOUT provides effective unsupervised on-chip outlier detection performance with high accuracy levels (over 0.9) and with low area and low power overhead (2.2% die area overhead and 1% idle power consumption). Outlier diagnosis can identify the chip's status with classification accuracy and F1-scores that exceed 0.8.

Index Terms—Silicon Lifecycle Management, Hardware Telemetry, Outlier Detection, Memory-efficient

I. INTRODUCTION AND MOTIVATION

The deployment of integrated circuits for safety and mission-critical applications has led to increased interest in an ecosystem of in-silicon (edge-based) and cloud-based solutions that offer increased product lifespan while maintaining quality of service [1]. Silicon lifecycle management (SLM) has emerged with the need to meet these goals [2]. This is achieved through an interplay between several architectural elements, which include the following. (1) *On-chip monitors and sensing IP* that collect silicon data on process characterization (PVT monitoring), power supply noise (voltage droop), path delay, and aging during the different stages of the product lifecycle, including fabrication, test, and in-field operation [3]–[5]. (2) A *telemetry architecture* that transmits data collected from the monitoring IP to on-chip (edge-based) compute units and enables subsequent diagnosis through a cloud-based service

[6]–[9]. (3) *Statistical and machine learning (ML)-guided decision models* that can derive valuable insights from the collected data [10]–[14].

SLM telemetry architectures may be used to better inform design decisions, reduce operational downtime, improve quality, and enhance in-field reliability [2], [6], [8], [9]. However, incorporating these SLM features may be an expensive undertaking due to the increased development cost. SLM IPs may compete with on-chip resources for limited power system budgets and area [15]. System-level integration of such components may lead to increased time to market. Furthermore, cloud-based supervised machine learning (ML) models must be trained on massive amounts of multi-feature labeled data, which is expensive and often infeasible. In addition, supervised models may need to be retrained to adapt to silicon aging during the product lifecycle [16], [17].

To address the above challenges, we present **E-SCOUT: Efficient-Spatial Clustering-based Outlier detection through Telemetry**. E-SCOUT operates at the interplay of SLM edge and SLM cloud. It provides efficient resource-constrained on-chip unsupervised outlier detection and highly accurate classification outlier diagnosis. In addition, E-SCOUT utilizes built-in system-level telemetry infrastructure (e.g., performance counters, on-chip sensors) to monitor silicon safety and security through a low-cost hardware/software co-designed outlier detection solution. The outlier detection mechanism does not require hard-to-get labeled data, thereby enabling unsupervised learning. Prior work has attempted this, but only at the software level [8], [9] and not as a hardware/software co-designed SLM framework. E-SCOUT can address in-field SLM issues such as detecting security- and safety-related threats. These objectives are achieved with low area overhead and power consumption. The key contributions of this paper are as follows:

- 1) A compute-efficient unsupervised outlier detection method that utilizes on-chip telemetry data to achieve various SLM goals in resource-constrained environments.
- 2) An unsupervised feature importance ranking method that enables feature selection for outlier detection.
- 3) Microarchitecture recommendations to implement E-SCOUT as a hardware/software co-designed SLM so-

lution.

- 4) Real telemetry silicon data is collected to detect security and safety threats (Rowhammer, SPECTRE, and voltage droops).

We demonstrate E-SCOUT's SLM effectiveness in detecting anomalous events relating to security and safety with high outlier detection performance with high accuracy levels that exceed 0.9. E-SCOUT achieves these goals while eliminating the need for labeled training data. We evaluate E-SCOUT's power, performance, and area (PPA) overhead through design-space exploration.

The remainder of the paper is organized as follows. In Section II, we discuss related prior work. In Section III, we describe the proposed method and implementation for E-SCOUT (SLM edge and SLM cloud). In Section IV, we discuss our experimental setup with real hardware. In Section V, we present results and subsequent analysis. Section VI concludes the paper.

II. BACKGROUND

A. Cross-Functional Goals driven by SLM and Telemetry

Different aspects of SLM and system-telemetry monitors enable designers to meet end goals targeted toward increased in-field system reliability, availability, and serviceability. Three end goals may be targeted at various product stages:

- 1) **Design Stage:** Design optimization targeted towards power, performance, and area optimization of individual standard cells.
- 2) **Test and Fabrication Stage:** Process improvement, DPPM (defective parts per million) reduction through test pattern development, yield ramp-up.
- 3) **Operational Stage:** Reduction in system downtime through improved functional resilience, silicon security, and silicon health monitoring.

Prior work has focused on developing solutions targeted towards improved system health/reliability for core router and memory-based subsystems [18]–[20]. In conjunction, the effectiveness of utilizing telemetry sensors to enhance system-level security [13], [21]. In addition, it has also been shown that telemetry sensors may enhance silicon safety through the prediction of thermal stress-induced silicon degradation and the minimum operating voltage [14], [22] has been demonstrated. Prior work has shown that dedicated hardware may be used for automotive SLM applications [23]. However, [23] requires dedicated microcontrollers to enable its SLM implementation and does not consider security threats. Unsupervised learning methods are useful for post-silicon bug diagnosis [24]. However, none of these prior methods consider the intersection of in-field security and safety monitoring through an unsupervised learning framework.

B. Outlier Detection & Diagnosis

Outlier detection refers to finding patterns that deviate from the expected (nominal) behavior [24]–[26]. Data patterns that deviate from the nominal behavior are referred to as

outliers. Outlier detection methods have been explored through unsupervised models such as iForest and autoencoders [24]–[26]. iForest is an ensemble method that creates randomly split binary search trees to characterize multi-feature data [25]. It has been shown to have high outlier detection performance with low memory cost [25]. However, iForest has inherent anomaly blind spots that may be adversarially exploited [27]. Autoencoders represent an unsupervised learning method built from neural network encoding and decoding [26]. They have also been shown to have high outlier detection efficacy [26]. However, neural network-based methods have memory requirements that are impractical for resource-constrained environments [28].

Outlier diagnosis has also been studied [27], [29]. For example, [27] describes an outlier-supervised version of Isolation Forest to improve performance. However, outliers seldom happen, therefore using multi-feature labels to characterize them is not practical. [29] presents a method based on outlier diagnosis and subsequent data filtering to improve dataset quality. This method [29] is specific for each dataset, requires labels, and is therefore not practical. Another approach utilizes anomaly data for supervised failure prediction but this is a software-based scheme [9].

III. PROPOSED METHOD: E-SCOUT

A. Block-Level Architecture

E-SCOUT includes an on-chip outlier-detection edge and an outlier-diagnostic cloud service. The outlier-diagnostic cloud consists of classification models that use the outlier detection scores of the E-SCOUT edge floating-point arithmetic operations to determine the state of the IC. A high-level representation of the proposed E-SCOUT architecture is shown in Figure 1. As seen in Figure 1, additional telemetry compute hardware is added to each chip IP for outlier event detection. When outlier events are detected, they are sent to the cloud for further diagnosis. The E-SCOUT architecture enables real-time in-field monitoring and it streamlines the necessary data analytics for SLM. Note that the outlier detection floating-point arithmetic operation is associated with each telemetry register per IP (i.e., model-specific registers) to ensure real-time (single cycle) latency for outlier detection. We define the specific operators and detection mechanisms in this section. In conjunction, we use an unsupervised feature importance ranking system to ensure that we use the essential minimum number of telemetry features for outlier detection. In addition, we provide microarchitecture recommendations for an E-SCOUT hardware/software co-design.

B. Outlier Detection Mechanism

The E-SCOUT outlier detection mechanism utilizes telemetry/sensor data for on-chip processing. We consider the telemetry/sensor chip data X as a multi-feature dataset, i.e. $X = \{X_{f1}, X_{f2}, \dots\}$. Each feature is a set of sample points, i.e., $X_{f1} = \{x_1, x_2, \dots\}$. To simplify notation, we use X_f to denote an arbitrary feature within X hereafter. Note that we define telemetry-based features X_f as a random variable

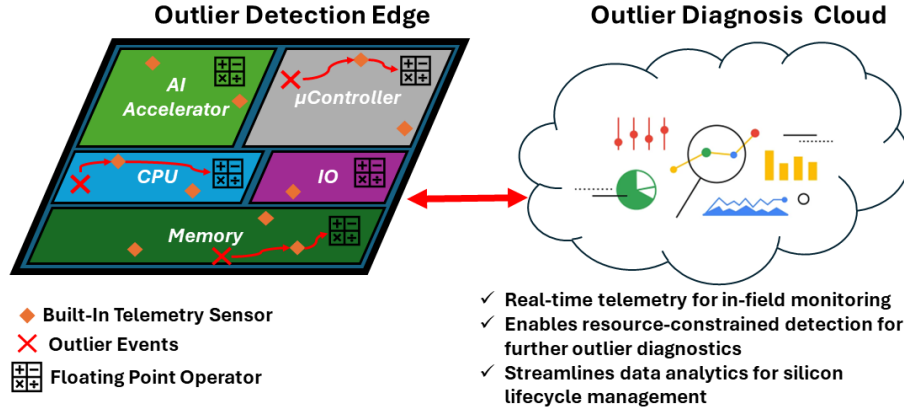


Fig. 1. Block-level diagram of proposed outlier detection telemetry architecture for silicon lifecycle management.

due to process variation, environmental factors, and workload variations. The expected value (mean) of X_f is denoted by μ_f . The standard deviation of X_f is denoted by σ_f . We use the loss function (i.e., $\|X\|_2^2$) as the basis for the outlier detection mechanism for E-SCOUT [30]. The loss functions are denoted by $L_1(X)$ and $L_2(X)$. We consider 16-bit and 32-bit floating point precision implementations (i.e., *float16*, *float32*) for each loss function (i.e., *float16* $L_1(X)$, *float16* $L_2(X)$, *float32* $L_1(X)$, and *float32* $L_2(X)$). Note that μ_f and X_f represent the expected and current/inference telemetry values from the system. The first proposed outlier detection ($L_1(X)$) is the sum of squared errors across all considered telemetry features (X_f).

$$L_1(X) = \sum_{X_f \in X} (X_f - \mu_f)^2 \quad (1)$$

The second proposed outlier detection, $L_2(X)$, is derived from the standardized error of X_f . Note that σ_f is the standard deviation of X_f . This proposed outlier detection method relies on the sum of standard deviations across all considered telemetry features (X_f).

$$L_2(X) = \sum_{X_f \in X} \left(\frac{X_f - \mu_f}{\sigma_f} \right)^2 = \sum_{X_f \in X} \left(\frac{1}{\sigma_f} (X_f - \mu_f) \right)^2 \quad (2)$$

We derive the specific number of considered telemetry features through unsupervised feature importance ranking as described next.

C. Unsupervised Feature Importance Ranking

We perform unsupervised feature importance ranking based on information entropy, defined in Equation (3). Given the probability distribution of X_f , we establish the average information content of an outcome [31]. Note that $\mathbb{P}[X_f]$ represents the probability mass function of X_f .

$$H(X_f) \equiv \sum_{x \in X_f} \mathbb{P}[x] \log_2 \frac{1}{\mathbb{P}[x]} \quad (3)$$

Note that previous work on unsupervised feature rankings based on information entropy aims to enhance down-

stream *supervised* tasks [32]. E-SCOUT's unsupervised ranking methodology is aimed at enhancing downstream *unsupervised* outlier detection. Each X_f is scored based on its information content (i.e., $H(X_f)$). The features are then ranked in descending order by their resultant information entropy, i.e., the top feature has the most information content. We sweep the number of top features and show the resulting outlier detection performance. We found that utilizing only the top 50% information-rich features was sufficient to provide effective outlier detection and subsequent outlier diagnostics. We describe the type of features collected in Section IV. Outlier detection performance results are shown in Section V.

D. Reuse of Model-Specific Register Space

E-SCOUT's outlier detection is derived from analyzing telemetry features through performance counters and built-in sensors. These may be accessed through the 64-bit wide model-specific register (MSR) space [33]. In conjunction, each MSR has an address that is 32-bits wide [33]. Each performance counter and sensor may be accessed through general-purpose 32-bit registers such as *ECX*, *EDX*, and *EAX*. For example, the instruction *RDMSR/WRMSR* may be used to read/write to a specific MSR at address *ECX* [33]. The general-purpose registers *EDX:EAX* may be used to denote the 64-bit value to write to MSR at address *ECX* [33].

Note that SCOUT's outlier detection has been defined by $L_1(X)$ and $L_2(X)$. $L_1(X)$ considers $-\mu_f$ as a parameter, while $L_2(X)$ considers $-\mu_f$ and $\frac{1}{\sigma_f}$ as parameters. To enable the parameters of these outlier detection mechanisms to be computed on-chip, we repurpose MSR registers that are reserved for extended features, i.e., *IA32_EFER* [33]¹. Figure 2 shows the currently defined extended feature MSR space. We use a one-hot encoding scheme to enable hardware/software integration of E-SCOUT through the extended feature MSR space. For example, *IA32_EFER* bit 1 (2) may act as a control bit to enable write access for MSR parameter $-\mu_f$ ($\frac{1}{\sigma_f}$) at address *ECX*. When these no bits are set, e.g., control bits are all set to '0', then the *WRMSR* instruction will write to MSR at address *ECX*. In addition, we may use

¹For further information on reserved MSR registers for extended features, see the Intel architecture manual Volume 3A, Chapter 2, Section 2.2.1 [33]

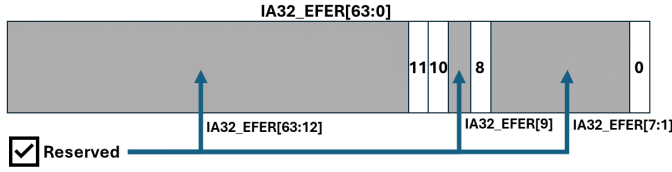


Fig. 2. IA32_EFER register space.

bits IA32_EFER[32:63] to store the real-time outlier score (assuming 32-bit precision).

In total IA32_EFER bits 1 through 7, 9, and 12 through 63 are reserved for additional features to be added to the micro-architecture [33]. This implies that up to 60 potential parameters may be used for additional telemetry-based on-chip processing. E-SCOUT requires 34 bits of the IA32_EFER space (2 bits for parameter write control access, 32 bits for outlier scoring). The design parameters for outlier analysis can be adjusted in-field with low latency. Accessing MSRs from a user-space process takes 5000 cycles (approximately 1.1 μ s for a 4.2 GHz clock frequency) [34]. For kernel-level implementations, this latency may be lowered to 100 cycles (approximately 23.8 ns for a 4.2 GHz clock frequency). Utilizing unused and built-in register space, E-SCOUT provides the necessary in-field SLM extended features to enable real-time safety and security monitoring.

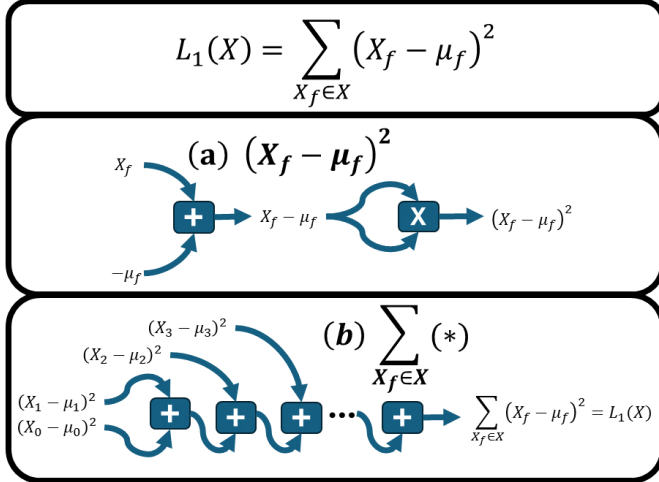


Fig. 3. Hardware block-level diagram of the $L_1(X)$ outlier detection mechanism.

E. Hardware Implementation

The telemetry feature selected for outlier detection and the corresponding E-SCOUT parameters act as input into the hardware implementation of the proposed outlier detection method. All on-chip outlier detection mechanisms are based on floating-point arithmetic (IEEE 754) [35]. The $L_1(X)$ and $L_2(X)$ outlier detection mechanism hardware block-level diagrams are described in Figure 3 and Figure 4, respectively. In Part (a) of each figure, the relative error is computed per telemetry feature X_f , e.g., the squared error is computed in Figure 3(a), and the standardized error is computed in Figure 4(a). In Part (b) of each figure, the telemetry feature errors

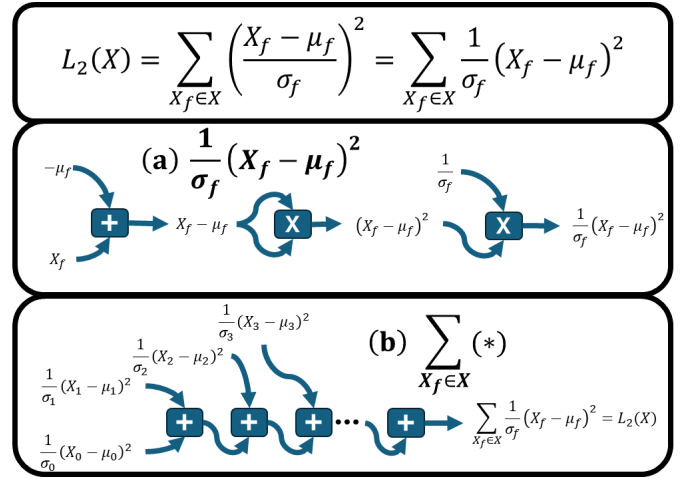


Fig. 4. Hardware block-level diagram of the $L_2(X)$ outlier detection mechanism.

are summed to compute the outlier score. Note that computing the error for all telemetry features and performing subsequent summation is expensive and not practical. To ensure lower area overhead, we only implement the necessary hardware for each telemetry feature based on its information content. Our unsupervised feature importance ranking informs our design decision to implement the necessary hardware on the top 50% information-rich telemetry features. Our results are shown later in Section V.

F. Outlier Diagnosis Cloud

As seen in Figures 3-4, the outlier detection scoring sums the error from the expected state of the IC into a singular feature vector used for outlier diagnostics. These single-feature vectors are used to train the outlier diagnosis models for the E-SCOUT SLM cloud. This removes the need for expensive multi-feature labels for in-field IC diagnostics. We consider multiple classification models for our outlier diagnostics cloud. Prior work also considers this approach [8], [9], but through a software-only approach.

We aim to classify three in-field IC states. The first is a benign state, when the suspicious event is recorded and the normal workload is executed. The second is a security threat when a malicious program is running in the background of the workload. The third is a safety risk for the IC such as voltage droop that may occur due to environmental factors.

IV. EXPERIMENTAL SETUP

We have developed experimental setups to cross-validate the proposed E-SCOUT architecture. The configurations for these experimental setups are shown in Table I. In addition, we provide pictures of our experimental setups in Figure 5. Note that each setup utilizes different hardware as seen in Table I and Figure 5. Note that each MSR has a specific 32-bit address. Hence, there may be up to 2^{32} MSRs. A vast majority of all MSRs are either unused or act as control registers for the greater system [33]. We do not collect all MSR information as this is exhaustive. To enable outlier-based

TABLE I
EXPERIMENTAL SETUPS USED TO VALIDATE E-SCOUT OUTLIER DETECTION.

SPECIFICATIONS	SETUP A	SETUP B
LINUX VERSION	UBUNTU 20.04.6 LTS (FOCAL FOSSA)	UBUNTU 20.04.6 LTS (FOCAL FOSSA)
KERNEL	5.15.0.101-GENERIC	5.15.0.101-GENERIC
CPU	i5-7600K	i5-12600K
NUMBER OF CORES	4	16
L1 SIZE	128 KB	240 KB
L2 SIZE	1 MB	6.3 MB
L3 SIZE	6 MB	N/A
DRAM (GENERATION, SIZE)	CMK8GX4M2A2666C16 (DDR4, 4×4 GB)	K23GX5M2A4800C40 (DDR5, 2 × 16 GB)
CLK FREQUENCY	4200 MHz	4500 MHz
CPU VOLTAGE (V_{DD})	1.284 V	1.128 V
STORAGE	SAMSUNG SSD 970 EVO PLUS 500 GB	SAMSUNG SSD 980 PRO 1 TB
MOTHERBOARD	GIGABYTE Z270XP-SLI-CF	ASUS PRIME Z690-A
LITHOGRAPHY SIZE	14 nm [36]	7 nm [37]
DIE AREA (mm^2)	125 mm^2 [38]	215.25 mm^2 [39]

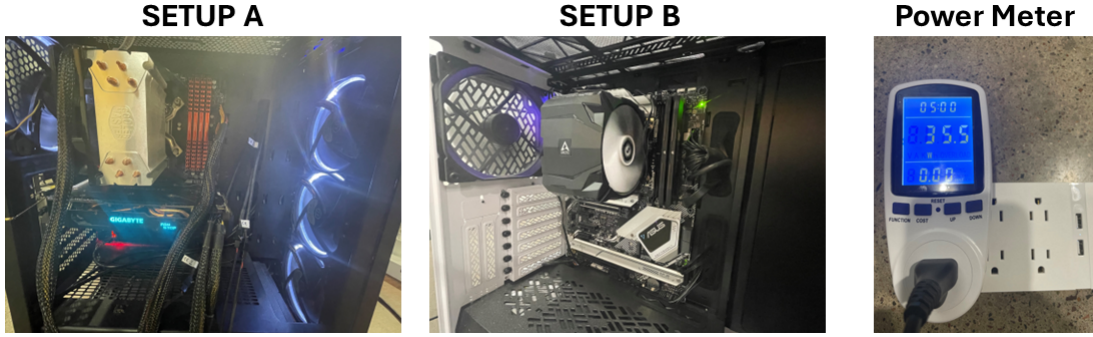


Fig. 5. Hardware setup for E-SCOUT data collection and emulation.

observability, we only collect MSR information from system-level outputs such as performance counters and sensors. The MSR-specific performance counters and telemetry sensors we collect come from open-source tools such as Intel’s PCM, Im_sensors, and voltage readings through MSR 0x198 [40]–[43]. The total number of features we collected for Setup A and B is 181 and 460, respectively. The collected telemetry data for Setup A is only $4.214 \times 10^{-6} \%$ of the total IA32 MSR space. For Setup B, the collected telemetry data is only $1.071 \times 10^{-5} \%$ of the IA32 MSR space. We only use 50% of the collected telemetry features for outlier detection, i.e., only $2.107 \times 10^{-6} \%$ ($5.355 \times 10^{-6} \%$) of the MSR space is used for on-chip outlier detection in Setup A (B).

Each benchmark from the PAMPAR suite [44] is run on each setup. We trace all applicable telemetry counters/sensors during PAMPAR benchmark workloads to capture the telemetry signature during runtime. We use acronyms for each PAMPAR benchmark; the benchmarks are listed in the footnote². We collect telemetry data while executing our benchmarks to build our “nominal” signatures, i.e., the parameter inputs designed for E-SCOUT. This nominal signature also represents our first classification class for outlier diagnostics.

We repeat our experiment while injecting outlier activity. The injected activity is based on two in-field SLM domains — security and safety. The first anomalous activity that we

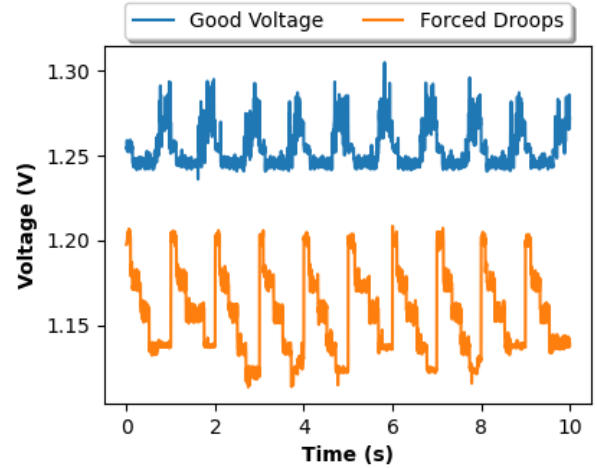


Fig. 6. Normal voltage readings from setup B and forced voltage droops. The voltage droops were forced through a modified version of plundervolt [42], [43].

consider is aimed at characterizing the telemetry signatures in the presence of a security attack. We use two separate attacks for each setup. On Setup A (B), we use TRRespass (Spectre) to flip bits (leak information) during the execution of each PAMPAR benchmark workload. The signatures collected during these security attacks represent the second classification class for outlier diagnostics. The next anomalous activity is aimed at identifying telemetry signatures during voltage-based safety risks. We modify a known safety vulnerability called Plundervolt to cause these voltage droops in real-time

²II calculation (PI), Dot product (DP), Numerical Integration (NI), Odd-even sort (OE), Harmonic Sums (HA), Discrete Fourier transforms (DFT), Matrix multiplication (MM), Gram Schmidt (GS), Jacobi method (JA), Dijkstra (DJ), Turing ring (TR), Histogram similarity (SH), Game of Life (GL)

[42], [43]. This vulnerability has been patched since it was identified; however, we use it to forcibly cause voltage-droops events [42]. As seen in Figure 6, voltage droops are forced on Setup A during runtime for the PAMPAR DFT benchmark. Note that we consider and run all PAMPAR benchmarks in the presence of forced voltage droops. We refer to these voltage-droop safety incidents as V-DROOP hereafter. We experimentally found that a +15% voltage droop can lead to a system crash.

In conjunction, we also collect overall power consumption during the execution of each PAMPAR workload that we considered. We use a power meter to identify the power consumption of our systems when idle versus when PAMPAR workload benchmarks are running and anomalous events are injected. The power meter that we use for our experimental setup can be seen in Figure 5 [45].

TABLE II
AREA AND POWER FROM SYNOPSIS DESIGN COMPILER [46].

OPERATOR	PRECISION (BITS)	AREA (μm^2)	POWER (mW)
MULTIPLIER	32	5280.99	0.56
MULTIPLIER	16	1318.27	0.1081
ADDER	32	4300.98	0.2272
ADDER	16	1660.26	0.1063

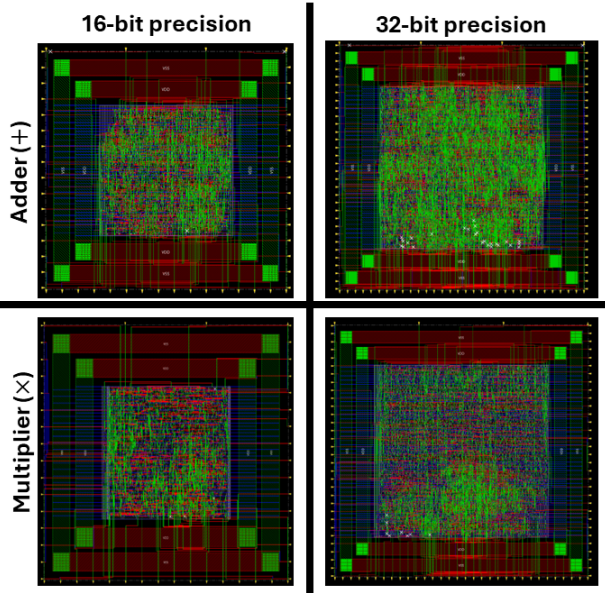


Fig. 7. Layout of the adder (top row) and multiplier (bottom row) for both 16-bit (left column) and 32-bit precision (right column).

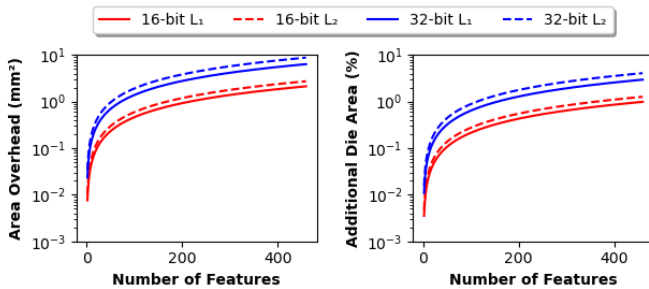


Fig. 8. Area (mm^2), and additional CPU die area (%) results for each configuration of E-SCOUT.

V. RESULTS & ANALYSIS

A. Power and Area Overhead

Note that each considered and collected telemetry feature is mapped to dedicated hardware to compute the telemetry feature error, as shown in Figures 3-4. The E-SCOUT outlier score is the summed telemetry feature error. Note that floating-point arithmetic is used to compute the outlier scores to ensure the desired level of accuracy in outlier detection. Hence, each considered outlier detection score is computed with floating-point arithmetic blocks, e.g., floating-point adder and floating-point multiplier. The area and power estimations for the floating-point arithmetic are obtained through the Synopsys Design Compiler, as seen in Table II³ [46]. The layout of each floating-point arithmetic is obtained through Cadence Innovus as shown in Figure 7.

The synthesis and layout are performed with the OPENROAD ASAP7 PDK library (7 nm) [47]. Note that Setup B is based on a system that uses the Intel 7 nm node. Setup A is based on a system that uses the Intel 14 nm node. Hence, we directly compare our area and power results with Setup B and not Setup A due to lithography size differences. We sweep the number of considered telemetry features for Setup B and provide the estimated area overhead in Figure 8. The area overhead is calculated using the die area for the processor used in Setup B.

Note that the area overhead and total power consumption are directly correlated to the number of considered outlier detection telemetry features. In addition, the power consumption also depends on the clock frequency of the E-SCOUT edge. Increasing the number of considered outlier detection telemetry features increases overall power consumption. In addition, an increased clock frequency of the E-SCOUT edge increases power consumption but lowers execution time for outlier detection. Hence, we sweep the E-SCOUT edge clock frequency and outlier detection telemetry features to understand the PPA tradeoffs. An E-SCOUT configuration is the number of considered telemetry features used for outlier detection with a specific clock frequency. The power consumption results (mW) and idle per consumption results (%) for all configurations of the E-SCOUT edge are shown in Figures 9 and 10, respectively.

As seen later in this section, E-SCOUT provides excellent outlier detection and diagnostics when the top 50% telemetry features are selected. With the top 50% telemetry features selected for on-chip outlier detection, the overall SCOUT area overhead (power consumption overhead) is 2.2% (1%). Note that when the considered PAMPAR benchmarks are running, the system power consumption increases, compared to the idle power ratings that were used to compute the overhead in Figure 10. Hence, the E-SCOUT power consumption overhead during workload execution is lower as seen in Figure 11.

³The clock frequency for the synthesized floating-point arithmetic is 900 MHz in Table II.

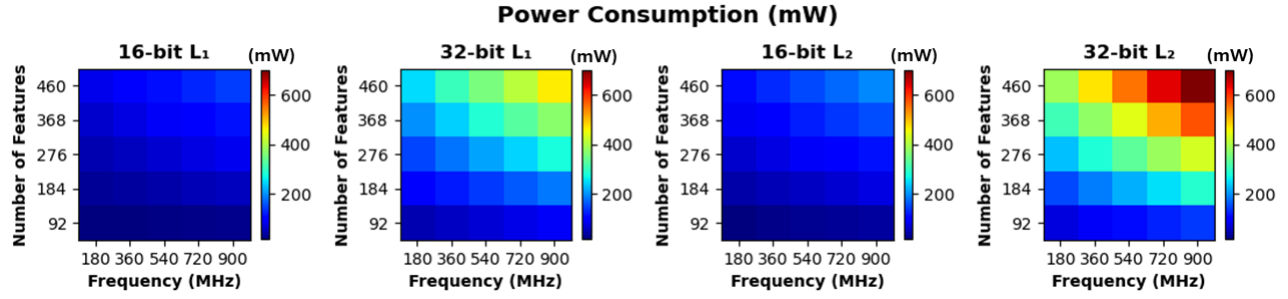


Fig. 9. Power consumption (mW) overhead for various configurations of E-SCOUT outlier detection. The X-axis (Y-axis) is the clock frequency (number of outlier detection telemeter features) of the E-SCOUT edge.

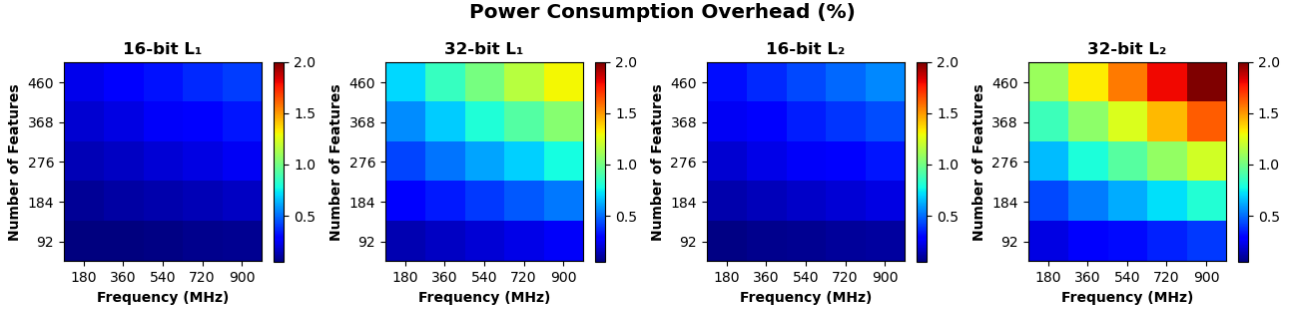


Fig. 10. Power consumption (%) overhead for various configurations of E-SCOUT outlier detection. The X-axis (Y-axis) is the clock frequency (number of outlier detection telemeter features) of the E-SCOUT edge. Note that the baseline is for an idle system, therefore the percentage overheads reported here are higher than what would actually be the case.

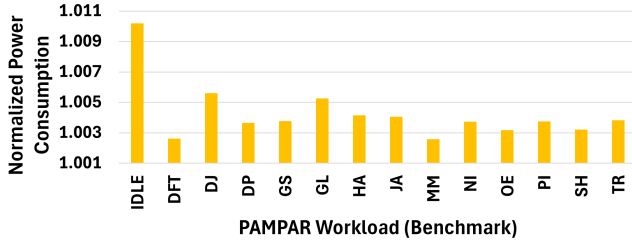


Fig. 11. Normalized power consumption overhead of the E-SCOUT edge. The clock frequency is 900 MHz and only the top 50% of collected telemetry features are selected for on-chip outlier detection. Normalization is carried out concerning the scenario when real-time anomaly detection at the edge is not carried out.

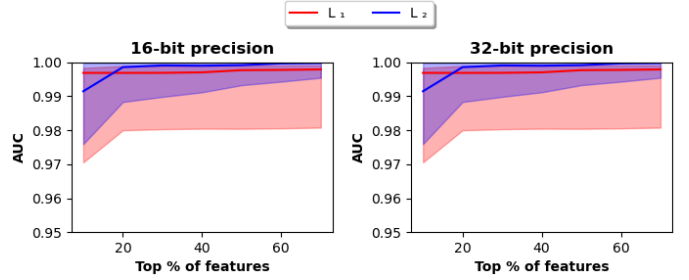


Fig. 13. E-SCOUT AUC scores for all considered workloads for the SPECTRE security attack. The y-axis (x-axis) is AUC (the top % telemetry features).

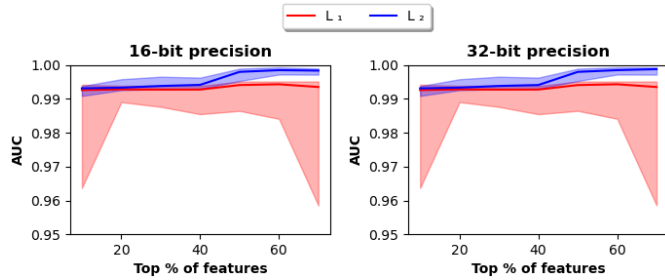


Fig. 12. E-SCOUT AUC scores for all PAMPAR workloads for the TRRespass security attack. The y-axis (x-axis) is AUC (the top % telemetry features).

B. Comparison to all loss functions

The number of available telemetry features collected is 181 (460) for SETUP A (B). We sweep the considered telemetry features from the top 10-70% of features for on-chip outlier detection (E-SCOUT edge). We use the Area

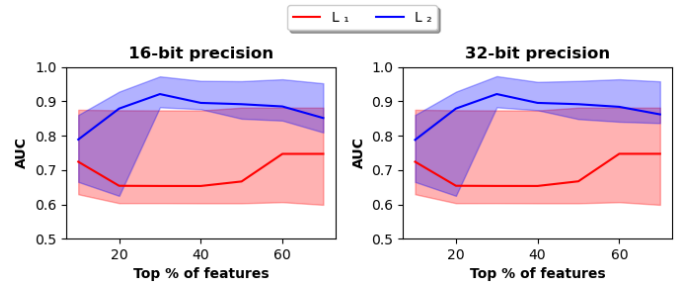


Fig. 14. E-SCOUT AUC scores for the PAMPAR workloads for the voltage droop safety incident on SETUP B. The y-axis (x-axis) is AUC (the top % telemetry features).

Under the Curve (AUC) as the metric to determine the outlier detection performance of the E-SCOUT outlier functions (i.e. $L_1(X)$, $L_2(X)$). The AUC is a commonly used measure of accuracy for inferencing methods. AUC values close to 1 are an indicator of effectiveness. All results are shown

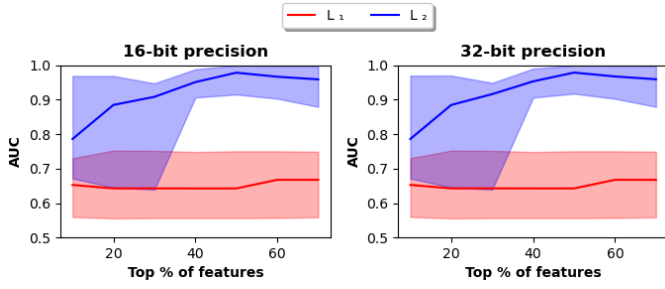


Fig. 15. E-SCOUT AUC scores for all considered workloads for the voltage droop safety incident on SETUP A. The y-axis (x-axis) is AUC (the top % telemetry features).

in Figures 12-15. Note that in all subplots in these figures, the x-axis is the % of top telemetry features considered for outlier detection. The top % of telemetry features are determined by the information entropy-based unsupervised feature importance ranking as stated in Section III. In these figures, the left (right) subplot shows the $L_1(X)$ and $L_2(X)$ results with 16-bit (32-bit) precision. Note that the blue (red) line indicates the $L_2(X)$ ($L_1(X)$) median AUC scores. We consider the distribution of AUC scores across all PAMPAR benchmarks. The shaded region represents the distribution of AUC scores from the 25th to 75th percentile of all computed AUC scores.

Figures 12-13 show the AUC scores obtained for the TR-Respass (SPECTRE) security attack on SETUP A (B). For the security-based outlier events, $L_1(X)$ and $L_2(X)$ perform well with high AUC scores. However, $L_2(X)$ outperforms $L_1(X)$ when more telemetry features are considered for on-chip outlier detection ($> 50\%$). A majority of all PAMPAR workload benchmarks have AUC scores above 0.97 for $L_1(X)$ and 0.99 for $L_2(X)$. Figure 14 (15) shows the AUC scores for the voltage droop safety incident on SETUP A (B). For safety-based outlier events, the $L_2(X)$ outlier detection mechanism outperforms $L_1(X)$ as the % of top telemetry features are considered. As seen in Figures 14-15, the $L_1(X)$ AUC scores consistently range between 0.6-0.9 (median is about 0.65). However, the $L_2(X)$ AUC scores improve as the number of telemetry features increases. In Figures 14-15, the AUC scores range from 0.8 to near 1.0 when the top 50% information-rich telemetry features are selected (the median AUC score is around 0.9).

From these results, we conclude that the $L_2(X)$ on-chip outlier detection scheme is the preferred choice for the E-SCOUT edge for our test cases. For the remainder of this paper, we consider the 32-bit $L_2(X)$ scheme as it offers greater precision than the 16-bit counterpart at only slightly higher overhead.

C. Comparison with State-Of-The-Art Unsupervised Learning Techniques

We compare E-SCOUT's outlier detection method with the two popular unsupervised methods: iForest and an autoencoder [25], [26]. All considered unsupervised learning models were implemented in Python. The ensemble iForest method consists

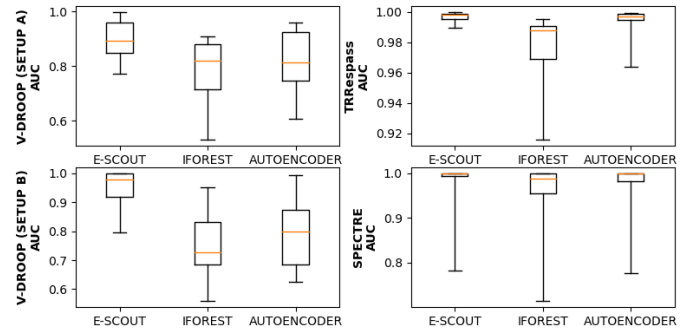


Fig. 16. Comparison of E-SCOUT, iFOREST, and AUTOENCODER AUC ranges for all PAMPAR workloads. Precision is 32 bits and the top 50% of telemetry features are selected for outlier detection.

TABLE III
MEMORY REQUIREMENTS FOR 32-BIT PRECISION OUTLIER DETECTION WITH E-SCOUT AND POPULAR UNSUPERVISED LEARNING TECHNIQUES (I.E., AUTOENCODER AND iFOREST). THE TOP 50% TELEMETRY FEATURES ARE SELECTED FOR MEMORY REQUIREMENT ESTIMATION

OUTLIER (SETUP)	E-SCOUT	AUTOENCODER	iFOREST
TRRESPASS (A)	0.72 KB	982.66 KB	220.00 KB
SPECTRE (B)	1.84 KB	1808.10 KB	194.80 KB
V-DROOP (A)	0.72 KB	982.66 KB	222.97 KB
V-DROOP (B)	1.84 KB	1808.10 KB	192.72 KB

of 100 trees and a sub-sampling size of 256. For the autoencoder, we utilized a batch size of 512, ran training for 100 epochs, and utilized six hidden layers for encoding (e.g., 64, 32, 16, 8, 4, 2) and five hidden layers for decoding (e.g., 4, 8, 16, 32, 64). To compare E-SCOUT's outlier detection to these state-of-the-art techniques, we train each model using the top 50% telemetry features for each benchmark. We implement a 32-bit floating point for all models. For each model, we compare outlier detection performance through AUC and memory usage in terms of the number of KBs needed to run the model.

We use AUC to quantify the outlier detection performance for each model. For conciseness, we report all the AUC score distributions via boxplot. As seen in Figure 16, each model has high AUC performance for all considered security and safety use cases. Note that for security use cases, the E-SCOUT outlier detection outperforms the iForest method and is on par with the autoencoder. For safety-based use cases (detecting voltage droop events), the E-SCOUT outlier detection mechanism outperforms the iForest and autoencoder methods. Overall, the E-SCOUT outlier detection is either at par with or can outperform the state-of-the-art unsupervised learning methods.

We next evaluate the memory usage for each outlier detection technique. The memory usage for E-SCOUT, iForest, and autoencoder are denoted in Table III. For E-SCOUT, the expected memory usage is based on the number of telemetry features selected for outlier detection. Note that the telemetry feature parameters (μ_f and σ_f) act as the memory usage for E-SCOUT. As we use a 32-bit implementation of the outlier detection mechanism, each parameter will consist of 4 bytes.

For E-SCOUT, the expected memory requirement for SETUP A (B) is 0.72 (1.84) KB. As for the other considered models, iForest and autoencoder require more memory than E-SCOUT. In SETUP A (B), E-SCOUT uses roughly $307\times$ ($104.5\times$) less memory than iForest. In SETUP A (B), E-SCOUT uses $1364\times$ ($982\times$) less memory than an autoencoder. Overall, E-SCOUT outperforms state-of-the-art unsupervised learning methods with significantly less memory usage.

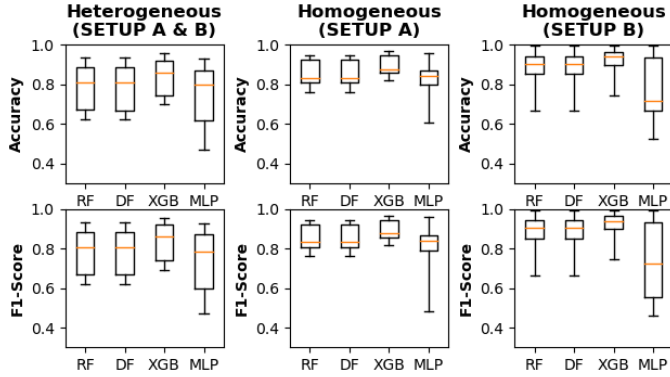


Fig. 17. E-SCOUT outlier diagnosis analysis. Each considered model is trained on E-SCOUT outlier detection scoring. A 70/30 split was considered for training/testing sets.

D. Cloud Outlier Diagnosis Results

From the resultant outlier scores from E-SCOUT (32-bit $L_2(X)$), the outlier diagnosis model is trained using the hardware's on-chip outlier scores. The outlier diagnosis models are classifiers that are tasked to understand which outlier scores align with which IC state (class). The classification models selected for comparison are Random Forest, Decision Forest, XG-Boost, and Multi-Layer Perceptron. Note that acronyms are used that are denoted in the footnote⁴. All models are implemented with their default settings from the SKLEARN library [48]–[51]. For E-SCOUT's outlier diagnostics, we compare each classifier's performance trained on on-chip outlier detection scores. Note that we consider two scenarios for training the outlier diagnostics cloud model. The first is an outlier diagnosis model trained on outlier scores from an E-SCOUT edge implementation on similar or uniform hardware. We refer to this type of training as homogeneous training. The homogeneous training we consider is E-SCOUT cloud models trained exclusively on SETUP A or SETUP B (but not both). The second is an outlier diagnosis model trained on outlier scores from E-SCOUT edge implementations on different or non-uniform hardware. We refer to this type of training as heterogeneous training. The heterogeneous training we consider is E-SCOUT cloud models trained from outlier scores from both Setup A and Setup B. We use classification accuracy and F1-score to characterize the E-SCOUT diagnosis cloud classification performance. We perform our analysis across all considered PAMPAR workloads.

⁴Decision Forest (DF), Random Forest (RF), XG-Boost (XGB), Multi-Layer Perceptron (MLP)

The results for outlier diagnosis are presented in Figure 17. The boxplot distribution of accuracy/F1-scores are shown across all PAMPAR benchmarks, i.e., the orange line is the median accuracy/F1-score result, the box shows the inter-quartile range, and the whiskers show the distribution range. The outlier diagnosis trained from heterogeneous (homogeneous) outlier scores is shown in the left (middle and right) column in Figure 17. The results for accuracy (F1-score) are shown in the top (bottom) row in Figure 17. As seen in Figure 17, all the models for all considered training types have high accuracy and high F1-score for most workloads. This indicates that on-chip outlier detection and subsequent diagnosis can yield accurate in-field IC classification. However, note that the MLP performance is inconsistent when compared with the Forest-based ML classification techniques (e.g., RF, DF, XGB). Hence, for the E-SCOUT SLM cloud, we conclude that Forest-based ML techniques are better suited for our use-case scenario. Given the distributions of accuracy/F1-scores, the XGB classifier yields slightly better results than the other forest ML techniques that we considered. Hence, for the E-SCOUT cloud, we consider the XGB as the preferred choice to work in conjunction with the E-SCOUT SLM edge (on-chip 32-bit $L_2(X)$).

VI. CONCLUSION

Silicon lifecycle management (SLM) ensures silicon-product reliability and quality. Prior methods utilize off-chip or resource-intensive solutions to identify malware, diagnose bugs, and characterize silicon health metrics. These methods do not explore hardware/software codesign for SLM. We have presented E-SCOUT: Efficient-Spatial Clustering-based Outlier detection through Telemetry, a silicon lifecycle management (SLM) hardware/software codesign methodology aimed to bolster in-field observability for outlier detection and further diagnostics. E-SCOUT provides two novel solutions to improve SLM cloud-edge architectures.

The first SLM contribution is an on-chip compute-efficient unsupervised 32-bit floating point outlier detection method that is enhanced through unsupervised feature importance ranking (E-SCOUT edge). Outlier detection is implemented as a hardware/software co-design with low power consumption and low area overheads (2.2% area overhead and 1% power consumption overhead compared to the baseline when the system is idle). In addition, we impart microarchitectural recommendations to enable a hardware/software co-design of the E-SCOUT edge. The second SLM contribution aids in simplifying SLM cloud diagnostics built on top of the E-SCOUT edge (referred to as E-SCOUT cloud). The E-SCOUT SLM cloud is trained from outlier scores of telemetry data. All telemetry data is collected through the model-specific register space using open-source Linux tools and Intel's performance counter. We capture the telemetry signatures of the PAMPAR benchmark suite while outlier events occur (i.e., security attacks such as Rowhammer and SPECTRE, and voltage-droop safety incidents). E-SCOUT provides effective unsupervised on-chip outlier detection performance ($AUC \geq 0.9$) with low

area and low power consumption overhead. Outlier diagnostics can classify the chip's status with accuracy and F1-scores that exceed 0.8.

ACKNOWLEDGMENT

This research was supported by the National Science Foundation under grant no. CNS-2310142 and by the Semiconductor Research Corporation (SRC) under contract no. 3177.001.

REFERENCES

- [1] J. Rajski *et al.*, "The Future of Design for Test and Silicon Lifecycle Management," *IEEE Design & Test*, 2023.
- [2] R. Kashyap, "Silicon lifecycle management (SLM) with in-chip monitoring," in *2021 IEEE International Reliability Physics Symposium (IRPS)*, 2021, pp. 1–4.
- [3] A. Ghosh *et al.*, "Slew-rate monitoring circuit for on-chip process variation detection," *IEEE Trans. VLSI*, vol. 21, pp. 1683–1692, 2012.
- [4] A. Simevski *et al.*, "Low-complexity integrated circuit aging monitor," in *14th IEEE DDECS*. IEEE, 2011, pp. 121–125.
- [5] S. Dhia *et al.*, "On-chip noise sensor for integrated circuit susceptibility investigations," *IEEE Trans. IM*, vol. 61, pp. 696–707, 2011.
- [6] F. Su *et al.*, "Product Health Insights Using Telemetry," *IEEE Design & Test*, pp. 1–1, 2023.
- [7] —, "Unleashing the Power of Anomaly Data for Soft Failure Predictive Analytics," in *2020 IEEE ITC*, 2020, pp. 1–10.
- [8] —, "Improving Analog Functional Safety Using Data-Driven Anomaly Detection," in *2018 IEEE International Test Conference (ITC)*, 2018, pp. 1–10.
- [9] —, "Unleashing the Power of Anomaly Data for Soft Failure Predictive Analytics," in *2020 IEEE International Test Conference (ITC)*, 2020, pp. 1–10.
- [10] R. Kwasnick *et al.*, "Telemetry for system reliability," in *2019 IEEE IEDM*. IEEE, 2019, pp. 13–2.
- [11] S. Jin *et al.*, "Anomaly Detection and Health-Status Analysis in a Core Router System," *IEEE Design & Test*, vol. 36, pp. 7–17, 2019.
- [12] —, "Data-Driven Resiliency Solutions for Boards and Systems," in *2018 31st International Conference on VLSID and 17th International Conference on Embedded Systems*, 2018, pp. 244–249.
- [13] H. Kumar *et al.*, "XMD: An Expansive Hardware-Telemetry-Based Mobile Malware Detector for Endpoint Detection," *IEEE Trans. on IFS*, vol. 18, pp. 5906–5919, 2023.
- [14] M. Amiraski *et al.*, "Boreas: A Cost-Effective Mitigation Method for Advanced Hotspots using Machine Learning and Hardware Telemetry," in *2023 IEEE ISPASS*, 2023, pp. 295–305.
- [15] A. Pandey *et al.*, "High Speed IO Access for Test forms the foundation for Silicon Lifecycle Management," in *2022 IEEE ITC*. IEEE, 2022, pp. 656–660.
- [16] M. Alam, "Reliability-and process-variation aware design of integrated circuits," *Microelectronics Reliability*, vol. 48, pp. 1114–1122, 2008.
- [17] M. Straat *et al.*, "Supervised learning in the presence of concept drift: a modelling framework," *Neural Computing and Applications*, vol. 34, pp. 101–118, 2022.
- [18] S. Jin *et al.*, "Self-Learning and Efficient Health-Status Analysis for a Core Router System," *IEEE Trans. CAD*, vol. 39, pp. 1935–1948, 2020.
- [19] —, "Hierarchical Symbol-Based Health-Status Analysis Using Time-Series Data in a Core Router System," *IEEE Trans. CAD*, vol. 39, pp. 700–713, 2020.
- [20] C. Argyrides *et al.*, "Utilizing ECC Analytics to Improve Memory Lifecycle Management," in *2023 IEEE ITC*, 2023, pp. 383–387.
- [21] R. Elnaggar *et al.*, "Accurate and Robust Malware Detection: Running XGBoost on Runtime Data From Performance Counters," *IEEE Trans. CAD*, vol. 41, pp. 2066–2079, 2022.
- [22] Y. Yin *et al.*, "Domain-Specific Machine Learning Based Minimum Operating Voltage Prediction Using On-Chip Monitor Data," in *2023 IEEE ITC*, 2023, pp. 99–104.
- [23] K. Kandula *et al.*, "SLM Subsystem for Automotive SoC: Case Study on Path Margin Monitor," in *2023 IEEE ITC*, 2023.
- [24] A. DeOrio *et al.*, "Machine learning-based anomaly detection for post-silicon bug diagnosis," in *2013 DATE*, 2013, pp. 491–496.
- [25] F. T. Liu *et al.*, "Isolation Forest," in *2008 Eighth IEEE ICDM*, 2008, pp. 413–422.
- [26] P. Baldi, "Autoencoders, unsupervised learning and deep architectures," ser. UTLW'11. JMLR.org, 2011, p. 37–50.
- [27] P.-F. Marteau *et al.*, "Hybrid Isolation Forest - Application to Intrusion Detection," 2017.
- [28] S. Yao *et al.*, "DeepIoT: Compressing Deep Neural Network Structures for Sensing Systems with a Compressor-Critic Framework," in *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems*, ser. SenSys '17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: <https://doi.org/10.1145/3131672.3131675>
- [29] F. Ridzuan *et al.*, "Diagnostic analysis for outlier detection in big data analytics," *Procedia Computer Science*, vol. 197, pp. 685–692, 2022, sixth ISICO 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050921024133>
- [30] J. Liu *et al.*, "Multi-task feature learning via efficient l2, 1-norm minimization," in *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, ser. UAI '09. Arlington, Virginia, USA: AUAI Press, 2009, p. 339–348.
- [31] D. J. C. MacKay, *Information Theory, Inference & Learning Algorithms*. USA: Cambridge University Press, 2002.
- [32] V. M. Rao *et al.*, "Unsupervised feature ranking based on representation entropy," in *2012 1st International Conference on RAIT*, 2012, pp. 421–425.
- [33] "Intel® 64 and IA-32 Architectures Software Developer's Manual," <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, 2023, [Online; accessed 03-30-2024].
- [34] "Software Tuning, Performance Optimization Platform Monitoring," Community Forum on Intel Performance Monitors, 2018, [Online; accessed 03-30-2024].
- [35] "IEEE Standard for Floating-Point Arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.
- [36] "Intel® Core™ i57600K Processor," Intel Specification Sheet, 2017, [Online; accessed 03-30-2024].
- [37] "Intel® Core™ i512600K Processor," Intel Specification Sheet, 2021, [Online; accessed 03-30-2024].
- [38] "Intel® Core™ i57600K Processor," Wikichip Kaby Lake Microarchitecture, 2017, [Online; accessed 03-30-2024].
- [39] "Intel® Core™ i512600K Processor," i512600K wikichip specification sheet, 2021, [Online; accessed 03-30-2024].
- [40] "intel performance counter monitor," <https://github.com/intel/pcm>, 2023, [Online; accessed 03-30-2024].
- [41] "Im_sensors," Im sensors link, 2023, [Online; accessed 03-30-2024].
- [42] K. Murdock *et al.*, "Plundervolt: Software-based Fault Injection Attacks against Intel SGX," in *2020 IEEE SP*, 2020, pp. 1466–1482.
- [43] —, "Plundervolt: How a Little Bit of Undervolting Can Create a Lot of Trouble," *IEEE Security & Privacy*, vol. 18, pp. 28–37, 2020.
- [44] A. Marques Garcia *et al.*, "PAMPAR: A new parallel benchmark for performance and energy consumption evaluation," *Concurrency and Computation: Practice and Experience*, vol. 32, p. e5504, 2020, e5504 cpe.5504. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.5504>
- [45] "Upgraded Brighter LCD Display Night Vision Power Meter Plug, Power Consumption Monitor Energy Voltage Amps Electricity Usage Monitor, Overload Protection, 7 Display Modes for Energy Saving, Watt Meter," Power Meter Amazon Link, 2024, [Online; accessed 04-21-2024].
- [46] "Design Compiler," <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>, 2023, [Online; accessed 03-30-2024].
- [47] L. T. Clark *et al.*, "ASAP7: A 7-nm finFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002626921630026X>
- [48] "Random Forest Classifier," SKLEARN Random Forest Classifier, 2024, [Online; accessed 04-14-2024].
- [49] "Decision Tree Classifier," SKLEARN Decision Tree Classifier, 2024, [Online; accessed 04-14-2024].
- [50] "Gradient Boosted Classifier," SKLEARN Gradient Boosted Classifier, 2024, [Online; accessed 04-14-2024].
- [51] "Multi-Layer Perceptron Classifier," SKLEARN Neural Network MLP classifier, 2024, [Online; accessed 04-14-2024].